REPRODUCTION BRIEF · 25 SEPTEMBER 2026

Adiabatic axion-cloud framework

What is implemented, what has been checked, and how to begin reproducing the calculation.

We solve a sequence of approximately stationary, self-interacting scalar clouds on a fixed Kerr background. Each solution is labelled by the fundamental mode's outer amplitude A_0. Its profiles, frequency, energy and signed boundary fluxes supply the ingredients for slow evolution from energy balance.

The target is the method of Omiya, Takahashi & Tanaka, *Adiabatic evolution of the self-interacting axion field around rotating black holes*, arXiv:2201.04382v3, especially their stationary construction and Figure 2.

> The stationary solver and preliminary branches are implemented. Implicit branch derivatives and interpolation checks are also available. A validated integration of $A_0(t)$, saturation result, and high-amplitude spatial convergence are still outstanding.

Scope of the saved calculation

Quantity	Value / convention
Background	Fixed Kerr; M = 1, a/M = 0.99, Mμ = 0.42
Seed cloud	Fundamental radial mode, $l_0 = m_0 = 1$
Potentials	Cosine, attractive quartic, and sextic truncation
Retained (l,m) modes	(1,1), (3,1), (5,1), (3,3), (5,3), (5,5)
Spatial discretization	RK4 in r_*, step 0.1; endpoints −100 and 150; match 5
Angular quadrature	16 Gauss–Legendre polar nodes × 31 periodic azimuthal nodes
Accepted branch residual	Euclidean norm of matching mismatch / $A_0 \le 10^{-9}$

Read and reproduce in this order

Conventions and equations (p. 2) → matching and observables (p. 3) → saved results and checks (pp. 4–5) → setup and linear benchmark (p. 6) → branch and derivative recipes (p. 7) → remaining validation and code map (p. 8).

Snapshot: repository revision **bc2ab9f** (full hash on p. 8). Numerical summaries and plots were regenerated from the checked-in CSVs. No new stationary solve or time evolution was run to prepare this brief. Older narrative handoffs are superseded where the source or saved tables contain newer results.

Conventions and equations

The field φ is dimensionless (physical axion field divided by F_a); $V(\varphi) = \mu^2 U(\varphi)$. The implementation sets M = 1 and uses exp(–iωt), so positive Im ω indicates linear growth.

$$\phi = \sum_{n,l} R_{nl}(r; A_0) Y_{l,nm_0}(\cos\theta) e^{-in(\omega_0 t - m_0\varphi)} + \text{c. c.}$$

Here n labels a **temporal/azimuthal harmonic**, not a radial overtone: $\omega_n = n\omega_0$ and $m_n = nm_0$. Thus the m = 3 and m = 5 blocks oscillate at $3\omega_0$ and $5\omega_0$ when $m_0 = 1$. The nonlinear source reconstructs the real field at the reference phase t = 0 and projects it back onto the retained basis.

The real polar harmonics include the Condon–Shortley phase and satisfy $\int_{-1}^{1} Y_{lm} Y_{l'm}\, dx = \delta_{ll'}$. Azimuthal factors are exp(imϕ); their quadrature weights integrate dϕ/(2π). Restore the full 2π factor for physical angular integrals. The radial field R is the paper's R, with **no extra 1/r rescaling**.

Amplitude normalization

$$R_{11} \sim A_0 e^{-qr} r^{-1 + (2\omega_0^2 - \mu^2)/q}, \quad q = \sqrt{\mu^2 - \omega_0^2}, \quad \operatorname{Re} q > 0$$

A_0 is the decaying-tail coefficient, not the peak field. It is fixed real and positive in each nonlinear solve; the linear solve fixes it to one.

Projected radial equations

$$\Delta R_{nl}'' + \Delta' R_{nl}' + B_{nl} R_{nl} + a^2(\omega_n^2 - \mu^2) \sum_{l'} C_{ll'}^{m_n} R_{nl'} + N_{nl} = 0$$

$$B_{nl} = \frac{[(r^2 + a^2)\omega_n - am_n]^2}{\Delta} - \mu^2 r^2 + 2am_n\omega_n - a^2\omega_n^2 - l(l+1)$$

$\Delta = r^2 - 2r + a^2$, $\Delta' = 2r - 2$. The coupling $C^m_{ll'} = \int Y_{lm} x^2 Y_{l'm} dx$ preserves m and couples l to l and l ± 2. N is the projection of $\mu^2(r^2 + a^2x^2)[\varphi - U'(\varphi)]$, with the **plus sign** in the radial equation above.

Potential selector	U(φ)	φ – U′(φ)
`:cosine`	1 – cos φ	φ – sin φ
`:quartic`	$\varphi^2/2 - \varphi^4/24$	$\varphi^3/6$
`:sextic`	$\varphi^2/2 - \varphi^4/24 + \varphi^6/720$	$\varphi^3/6 - \varphi^5/120$

Use the same potential in matching, profile reconstruction and energy. The cosine implementation guards against small-field cancellation. Harmonic truncation remains a separate approximation even when the full cosine potential is used.

Implementation / evidence: docs/adiabatic-design.md; src/adiabatic/angular.jl; src/adiabatic/stationary/nonlinear.jl

Boundary matching and observables

Integrate through space, then match

$$\frac{dr_*}{dr} = \frac{r^2 + a^2}{\Delta}, \quad Q = \frac{dR}{dr_*}, \quad \frac{dQ}{dr_*} = -\frac{2r\Delta}{(r^2 + a^2)^2} Q - \frac{\Delta}{(r^2 + a^2)^2}(BR + \text{mixing} + N)$$

Pack the state as $[R_1, \ldots, R_N, Q_1, \ldots, Q_N]$. Simplify the product ΔB analytically before evaluating near the horizon. Classical RK4 advances all modes in r_*; this is a spatial integration, not physical time evolution.

Inner boundary: $r_\pm = 1 \pm \sqrt{(1 - a^2)}$, $\Omega_H = a/(r_+^2 + a^2)$. Enforce $R \propto \exp[-i(\omega_n - m_n\Omega_H)r_*]$ using an ingoing series in $r - r_+$ through order 3.

Outer boundary: $R \propto \exp(ik_n r) r^{-1+i(2\omega_n^2 - \mu^2)/k_n}$, with a $1/r$ correction series through order 7. Choose $k = iq$ with $\mathrm{Re}\, q > 0$ for a bound mode, or the outward-wave branch for a radiative harmonic. The exact $k = 0$ threshold is excluded.

Integrate from both boundaries to $r_* = 5$ and match **both R and Q**. Inward integration from the outer boundary helps control contamination by the unwanted growing bound solution.

Nonlinear solve and continuation

For N modes, the unknowns are complex ω_0, N horizon coefficients/A_0, and N − 1 free outer coefficients/A_0: 4N real coordinates (24 for six modes). Pack real and imaginary parts separately because the real-field source involves complex conjugation. Damped Newton solves the residual divided by A_0; the first outer coefficient stays fixed.

Use a secant predictor or cubic extrapolation from four distinct, spaced history points, then correct the proposed amplitude. Failed trials halve the amplitude step and are excluded from accepted data. Current campaign defaults allow 12 Newton corrections per attempt; cosine uses ForwardDiff, polynomial branches use centered differences. Historical rows were produced with several earlier settings.

Energy and signed fluxes

Reintegrate an accepted solution with `save_profile=true`. Evaluate the Kerr energy density at $t = 0$, including time, radial, angular, rotation and potential terms. Reverse the descending outer profile and apply composite Simpson quadrature separately on both radial intervals; each needs uniform spacing and an even number of intervals. Their sum is the stored `energy` $= E/F_a^2$ in $M = 1$ units.

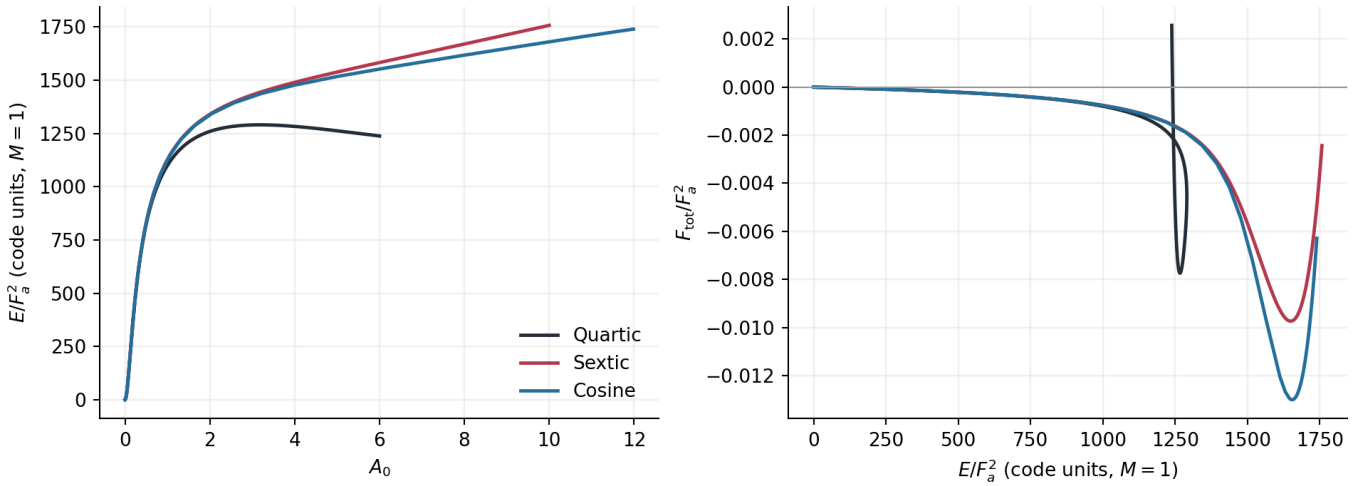
$$\frac{F_{\mathrm{out}}(r)}{F_a^2} = 4\pi(r^2 + a^2) \sum_{n,l} \mathrm{Re}\left[i\omega_n R_{nl} \overline{Q}_{nl}\right]$$

Store `horizon_flux` $= -F_{\mathrm{out}}$(inner), `outer_flux` $= F_{\mathrm{out}}$(outer), and `total_flux` = their sum. Negative total flux means cloud energy gain. These are finite-boundary currents. The diagnostic $-F_{\mathrm{tot}}/(2\,\mathrm{Im}\,\omega\, E)$ tends to one in the linear limit; do not impose it on nonlinear states.

Implementation / evidence: src/adiabatic/stationary/matching.jl; src/adiabatic/observables/energy.jl; runs/adiabatic/fig2_20260922/scripts/continue_cosine.jl

Saved stationary results

The three preliminary branches share a/M = 0.99 and Mμ = 0.42. The figure below merges production, refinement and gap tables, retaining production values at shared amplitudes. Curves connect accepted points in A_0 order.



Snapshot from the saved CSVs at revision bc2ab9f. Axes use the implementation's M = 1 energy and flux normalization. No smoothing, extrapolation or paper-curve digitization is applied.

Potential	Production / merged points	Last A_0	Last E/F_a^2	Last F_{tot}/F_a^2
Quartic	126 / 246	6.0000000	1238.260042	+0.0025691
Sextic	87 / 187	10.0000000	1757.555272	-0.0024324
Cosine	178 / 178	11.9851915	1739.893587	-0.0062873

All branches start at $A_0 = 0.0011390625$; all merged residuals are below 10^{-9}. The quartic energy reaches a sampled maximum near $A_0 = 3.18846$ ($E/F_a^2 \approx 1290.54$) and then turns downward. Sorting its flux curve by energy would scramble the branch. Cosine and sextic remain at negative total flux at their saved endpoints. Their requested continuation targets are not evidence that saturation was reached.

Completed checks supporting the framework

Linear frequency. The coupled l = [1,3,5], m = 1 solution gives
$M\omega = 0.408805564968254 + 1.503864692665 \times 10^{-7}$ i.
Recorded step, boundary and matching-point checks changed Re ω by less than 10^{-13} and Im ω by less than 2×10^{-16}. The single-l = 1 truncation was checked independently against Leaver with a consistent angular correction.

Linear sweep. Both radial overtones converged at 15 values of Mμ from 0.30 to 0.44 (30 points). At the two sweep endpoints, halving the step and increasing the outer boundary from 15 to 20 decay lengths shifted the growth rates by less than 4×10^{-9} fractionally.

Small-amplitude observables. Energy showed fourth-order radial refinement at $A_0 = 10^{-4}$. Moving the outer endpoint from 150 to 200 changed energy fractionally by 1.24×10^{-8}. The first production cosine point has a linear-balance ratio ≈ 0.999999895.

> These checks validate components and selected regimes. The displayed nonlinear curves have not yet passed a full high-amplitude radial, boundary, angular-quadrature and retained-mode convergence study.

Implementation / evidence: docs/adiabatic-repro.md; runs/adiabatic/fig2_20260922/README.md; runs/adiabatic/fig2_20260922/HISTORY.md

Preparing slow amplitude evolution

Finite differences or local polynomial fits to $E(A_0)$ can be inaccurate on unevenly spaced branches. We instead differentiate the discretized matching equations. Let x contain the real-packed frequency and normalized boundary coefficients, and f the scaled matching residual.

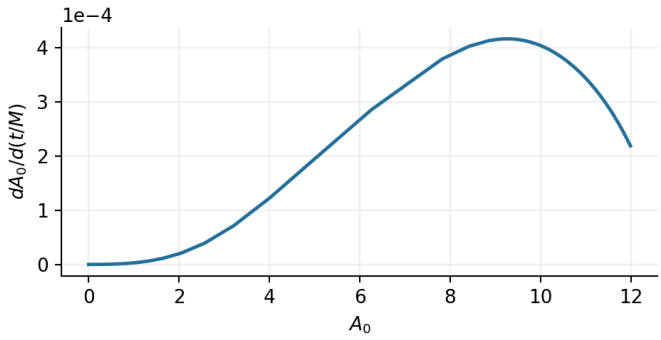
$$f(x, A_0) = 0, \qquad \frac{dx}{dA_0} = -\left(\frac{\partial f}{\partial x}\right)^{-1} \frac{\partial f}{\partial A_0}$$

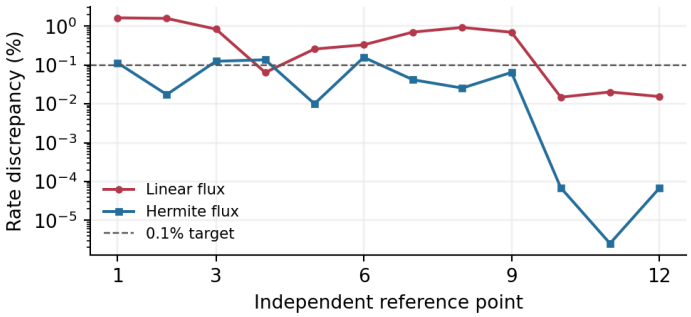
$$E' = \partial_{A_0} E + \nabla_x E \cdot x', \qquad \frac{dA_0}{d(t/M)} = -\frac{F_{\mathrm{tot}}}{E'(A_0)}$$

The code uses ForwardDiff to form the partial derivatives and solves a linear system for x′. It then differentiates energy along that direction, including explicit amplitude dependence. Physics and spatial settings stay fixed. The Jacobian must be nonsingular; this derivative measures the discretized branch, not its continuum error.

Saved derivative calculation: all 178 cosine amplitudes were rechecked/corrected to residual ≤ 10^{-10} (maximum 9.18428×10^{-11}; 135 points needed correction). Every saved E′ and amplitude rate is finite and positive. The largest sampled rate is 4.16088×10^{-4} at $A_0 \approx 9.23590$; the final rate is 2.18739×10^{-4}. These are instantaneous rates, not a trajectory.

Independent derivative check: at $A_0 \approx 2.564697905$, E′ = 80.7393443527. Separately corrected centered differences at spacings 0.025647, 0.0128235 and 0.00641174 disagree by 7.72×10^{-5}, 1.93×10^{-5} and 4.82×10^{-6} fractionally, showing the expected factor of four per halving.





Interpolation: latest saved results

Twelve independent reference solutions sample quarter, midpoint and three-quarter positions in four intervals. Cubic Hermite interpolation uses endpoint energies and E′. The initial rate implementation interpolates horizon and outer fluxes linearly. Newer saved tables also evaluate Hermite flux interpolation using flux derivatives.

Quantity compared with reference	Within 0.1%	Largest discrepancy
Hermite energy derivative	8 / 12	0.1570%
Rate with linear flux	4 / 12	1.6354%
Hermite total flux (newer saved check)	12 / 12	0.02059%
Rate with Hermite flux (newer saved check)	8 / 12	0.1574%

Improving flux interpolation helps substantially, but sparse-interval E′ errors still prevent the requested rate accuracy. The library's `amplitude_rate` currently uses linear flux. The newer flux tables are tracked; their generation driver is absent from this revision, so that experiment still needs a documented replay recipe. No time integration is implemented.

Implementation / evidence: src/adiabatic/amplitude_evolution/branch_sensitivity.jl; runs/adiabatic/amplitude_evolution/README.md; runs/adiabatic/amplitude_evolution/results/hermite_flux_checks.csv

Start reproducing: setup and linear seed

Use a separate checkout and run commands from its root. Keep the full source tree: Julia includes and saved campaign paths are relative to it. The repository may require access from its owner.

```
git clone https://github.com/clwelch03/axion-sr-sim.git
cd axion-sr-sim
git checkout bc2ab9f88f5ae0e867cf3607e2620c47190f0d2b
python -m pip install -e .
julia --project=. -e 'using Pkg; Pkg.instantiate()'
export OPENBLAS_NUM_THREADS=1
export JULIA_NUM_THREADS=1
```

Python requires ≥ 3.9 plus NumPy, SciPy, mpmath and Matplotlib. The Julia manifest records **Julia 1.13.0**, FastGaussQuadrature 1.3.0 and ForwardDiff 1.4.6. Use the recorded Julia environment when exchanging `.jls` files; do not assume serialization is portable across versions. Check `julia --version` and preserve the manifest.

First reproduce a frequency without a saved seed

Save this as `reproduce_linear.jl` in the checkout root:

```
include("src/adiabatic/load.jl")
grid = angular_grid(16, 31)
polar = build_polar_values(5, grid.cosine_theta_nodes)
couplings = build_cosine_squared_couplings(5, polar, grid)
solution = solve_linear_mode(
    0.4088 + 1e-7im, 0.99, 0.42, [1, 3, 5], 1, couplings;
    step=0.025, inner=-100.0, outer=100.0, match=5.0,
    tolerance=1e-12,
)
@show solution.frequency solution.residual_norm
@show solution.outer_amplitudes[1]
```

```
julia --project=. reproduce_linear.jl
```

Expect the frequency quoted on p. 4, positive Im ω, residual ≤ 10^{-12}, and first outer amplitude exactly 1. Compare the small imaginary part separately: agreement dominated by Re ω can hide an incorrect growth rate.

Run the component checks and initial nonlinear branch

```
julia --project=. tests/adiabatic/runtests.jl
julia --project=. src/adiabatic/drivers/continuation.jl
julia --project=. src/adiabatic/drivers/inspect_profile_agreement.jl
python src/adiabatic/plot_profiles.py
julia --project=. tests/adiabatic/check_nonlinear_convergence.jl
julia --project=. tests/adiabatic/check_energy_convergence.jl
```

The continuation driver derives its seed from the linear solution, starts at $A_0 = 10^{-4}$, and adds six steps multiplied by 1.5. It uses the six-mode cosine system, default radial step 0.05 and outer endpoint 100. Its seven points are a separate small-amplitude reference, not the production grid on p. 1. At the first point, the saved real frequency shift/A_0^2 is approximately −0.05508153.

The small-amplitude drivers replace corresponding reference outputs in the checkout. Run them in your reproduction copy. On Lawrencium, execute substantial solves and the full suite in a Slurm allocation; the commands above describe what to run inside that allocation.

Implementation / evidence: README.md; Project.toml; Manifest.toml; tests/adiabatic/test_matching.jl

Branch, plot and derivative recipes

Inspect the saved results first

```
python runs/adiabatic/fig2_20260922/scripts/plot_branch.py
python runs/adiabatic/amplitude_evolution/scripts/summarize_derivatives.py
```

These regenerate `plots/preliminary_fig2_20260922.{png,pdf}` and `plots/amplitude_growth_implicit_comparison.{png,pdf}`. The standard Figure 2 plot uses fixed paper-like axis limits; the overview on p. 4 uses limits covering all saved points.

Repeat a bounded cosine continuation in a separate output

This uses a frozen five-point seed (last $A_0 = 0.018225$) and re-solves subsequent points. Run inside an allocation or equivalent workstation job:

```
mkdir -p runs/adiabatic/fig2_20260922/work
export AXION_SEED_PATH=runs/adiabatic/fig2_20260922/seeds/preliminary_cosine_20260922.jls
export AXION_OUTPUT_PREFIX=runs/adiabatic/fig2_20260922/work/reproduction_cosine
export AXION_TARGET_AMPLITUDE=0.1
export AXION_RUN_SECONDS=1000
export AXION_JACOBIAN=forwarddiff
export AXION_PREDICTOR=secant
export AXION_NEWTON_MAX_ITERATIONS=12
julia --project=. runs/adiabatic/fig2_20260922/scripts/continue_cosine.jl
unset AXION_SEED_PATH
```

With the seed override unset, the same output prefix resumes its own checkpoint. A runtime budget is checked between attempts, so a solve can run beyond it. A target is a request, not a guaranteed completed endpoint. This recipe reproduces subsequent solves from a supplied seed; p. 6 builds a small-amplitude branch without one.

Quartic/sextic: use `AXION_POTENTIAL=quartic` or `sextic` with `scripts/continue_polynomial.jl` under the Figure 2 campaign. Its output path is fixed per potential. In a disposable reproduction checkout, move the corresponding `results/<potential>_branch.{csv,jls}` aside to start fresh; then it re-solves the first cosine seed for the chosen potential. Otherwise it resumes the saved branch. Use separate shells for independent recipes to avoid inherited overrides.

Recompute implicit derivatives independently

```
evolution=runs/adiabatic/amplitude_evolution
mkdir -p "$evolution/work"
julia --project=. src/adiabatic/drivers/branch_energy_derivatives.jl \
  "$evolution/seeds/cosine_branch_20260924.jls" \
  "$evolution/work/reproduction_derivatives" 0 1
```

The final arguments are a zero-based shard index and shard count. For four independent jobs, use indices 0–3, count 4 and a distinct output prefix for each. Restarting a prefix skips completed entries after checking the frozen-input hash and settings. The recorded four-shard campaign took about 8–13 minutes per one-CPU task; this is historical timing, not a portable estimate.

Repeat the independent single-point sensitivity check with `julia --project=. tests/adiabatic/check_branch_sensitivity.jl`. To assess the existing 12 interpolation references, run `julia --project=. tests/adiabatic/check_branch_interpolation.jl assess`. Preparation and retry procedures are documented in the amplitude-evolution README; preparation refuses to overwrite its frozen seed.

Slurm recipes live in each campaign's `scripts/`. Adapt the hard-coded Julia executable, account `pc_heptheory`, partition `lr6` and QOS `lr_normal` to your site. Create log directories before submission and use one writer per output prefix. CSVs store diagnostics; JLS files store restart states and settings.

Implementation / evidence: runs/adiabatic/fig2_20260922/README.md; runs/adiabatic/amplitude_evolution/README.md

Validation priorities and source map

What must be checked next

1. **High-amplitude spatial accuracy.** Re-solve shared amplitudes while halving radial spacing, moving both finite boundaries, changing the match location and varying boundary-series order. Compare energy, both signed fluxes, frequency and profiles. A smaller matching residual does not control these errors.
2. **Angular accuracy.** Increase quadrature nodes independently of the retained harmonic set, then enlarge the retained modes. Verify nonlinear-source projection and outgoing-harmonic flux. Filling A_0 gaps only refines branch sampling.
3. **Interpolation accuracy.** Add solved points in sparse intervals and repeat off-center reference checks. Preserve predictions before generating reference solutions. Package the saved Hermite-flux experiment into a reproducible driver. Check horizon and outer flux individually as well as their sum.
4. **Time evolution.** Once rates are accurate, integrate $dA_0/d(t/M) = -F_{tot}/E'$ within the validated branch range. Recover the linear limit $dA_0/dt \approx \mathrm{Im}\ \omega \cdot A_0$. Handle E' near zero and branch validity explicitly; the existing code does not cross or regularize turning points.
5. **Physical claims.** Compare converged curves with the paper before claiming saturation or collapse. Background spin-down, self-gravity and metric backreaction are not part of this fixed-Kerr calculation.

Where to read or modify the implementation

Location (under src/adiabatic/)	Responsibility
load.jl; angular.jl; tortoise.jl	Library entry point, angular basis, geometry
stationary/radial.jl; nonlinear.jl	Linear operator and projected self-interaction
stationary/boundaries.jl; boundary_series.jl	Ingoing / decaying / outgoing endpoint states
stationary/integration.jl; matching.jl	RK4 and frequency / boundary-amplitude solves
stationary/continuation_predictor.jl	Secant and guarded cubic guesses
observables/energy.jl	Finite-domain Kerr energy and Simpson integration
amplitude_evolution/branch_sensitivity.jl	Implicit state and energy derivatives
amplitude_evolution/interpolation.jl	Hermite energy and linear-flux amplitude rates
drivers/branch_energy_derivatives.jl	Corrected branch states, derivatives and flux tables

Campaign-specific continuation and flux routines are under `runs/adiabatic/fig2_20260922/scripts/`. Core tests are under `tests/adiabatic/`. `src/common/leaver.py` is the independent linear benchmark; Dolan's time-domain radial variables are not automatically interchangeable with this framework.

Provenance and references

Repository: github.com/clwelch03/axion-sr-sim
Revision: `bc2ab9f88f5ae0e867cf3607e2620c47190f0d2b`
Snapshot date: 25 September 2026. Working tree was clean before creating this PDF. Plots and counts were checked against saved tables; historical numerical validations were not rerun for this document.

Paper: H. Omiya, T. Takahashi & T. Tanaka, arXiv:2201.04382v3, method Eqs. (16), (20)–(23), (25), (29). Equations and implementation details in this brief follow the repository design and source, with the current finite-domain conventions.

Primary local records: design, reproduction status, Figure 2 campaign, numerical history, derivative/interpolation campaign. The newer `results/cosine_flux_derivatives.{csv,jls}` and `results/hermite_flux_checks.csv` in the latter campaign supply the updated flux-interpolation results on p. 5.